C를 이용한 웹 서비스 구현

간단한 형태의 IRC 서버 및 클라이언트 작성

Jonghyeon PARK

2026-01

General Exchange Student, Faculty of Science and Engineering, Saga University
25931028@edu.cc.saga-u.ac.jp; jonghyeon@jnu.ac.kr

웹 서버의 C 구현

과제 요구사항

- 클라이언트와 서버 모두에서 통신을 시작할 수 있어야 함
- 통신 프로토콜로서 TCP/IP(or UDP/IP) 사용
- 메시지를 수신하고 응답을 전송해야함.
  메시지는 프로그램에 사전 작성된 내용 혹은 직접 입력
- 서버와 클라이언트가 같은 PC에 있어도 됨

✓ 구현체: https://github.com/shapelayer/simple-irc

- 구현 내용
  -

서버/main.c

`main.c` / 프로그램 진입점: 메인 루프 호출 지점

```c
int main(int argc, char const **argv)
{
  // 명령줄 매개변수 파싱
  main_args_t args = parse_args(argc, argv);

  // 서버 세션 객체 생성
  server_sess_t *server_sess;
  server_sess = calloc(1, sizeof(server_sess_t));
  if (!server_sess) PANIC("Failed to allocate server session");
  init_server_sess(server_sess);
  printf("Server session initialized.\n");
```

서버/main.c

main.c

```c
// 서버 포트 설정 setter
server_sess_set_port(server_sess, args.port);
printf("Starting server on port %d...\n", args.port);
server_sess_open(server_sess);

// 메인 루프 진입
while (1) server_sess_update(server_sess);

return 0;
```

서버/server_sess.h

`server_sess.h` / 서버 세션 객체 선언

```
struct server_sess
{
  int fd;
  int opt;
  int sd_cnt;
  int port;
  struct sockaddr_in address;
  int addrlen;
  client_desc_t clients[];
  fd_set readfds;

  ...
} typedef server_sess_t;
```

`server_sess_t` 구조체의 내부 필드에 대해서

- ✓ `__` 로 시작하는 필드는 함수 내부에서 일시적으로 사용하는 값들
- ✓ `server_sess_t` 객체 자체와 밀접한 연관은 없음
- ✓ `update` 함수에서 매 회 생성하도록 해도 되지만, 매 회 변수 생성에 비용이 투입
- ✓ 최초 1회만 생성하여 변수를 재사용

서버/server_sess.h

server_sess.h

```c
struct server_sess
{
  ...
  // 내부 필드
  int __new_socket;
  int __valread;
  int __activity;
  char __str_buf[STRING_BUFFER_SIZE];
} typedef server_sess_t;
```

서버/server_sess.h

서버 세션 객체의 메서드 선언

```
// 생성자
void init_server_sess(server_sess_t *sess);

// 메인 루프
void server_sess_update(server_sess_t *sess);

// 서버 개방
void server_sess_open(server_sess_t *sess);

// 포트 설정
void server_sess_set_port(server_sess_t *sess, int port);

// 메시지 브로드캐스트
void server_sess_broadcast(server_sess_t *sess, int sender_idx, const char *msg);

// 클라이언트 메시지 처리
void server_sess_handle_client_message(server_sess_t *sess, int index, char *buffer);
```

서버/server_sess.c

```c
// 생성자
void init_server_sess(server_sess_t *sess)
{
  for (int i = 0; i < SERVER_MAX_CLIENT_CONNECTIONS; i++)
  {
    sess->clients[i].socket = 0;
    snprintf(sess->clients[i].nickname, sizeof(sess->clients[i].nickname), "Guest%d",
i);
  }

  // 소켓 개방 및 파일 디스크립터 할당 시도
  if ((sess->fd = socket(AF_INET, SOCK_STREAM, 0)) == 0)
  {
    PANIC("Error during opening the socket (socket)");
  }
...
```

서버/server_sess.c

```
...
  // 소켓 옵션 설정 시도
  sess->opt = 1;
  if (setsockopt(sess->fd, SOL_SOCKET, SO_REUSEADDR, &sess->opt, sizeof(sess->opt)))
  {
    PANIC("Error during setting the socket's option (setsockopt)");
  }

  // 서버 주소, 포트 데이터 설정 시도
  sess->address.sin_family = AF_INET;
  sess->address.sin_addr.s_addr = INADDR_ANY;
  server_sess_set_port(sess, SERVER_PORT);
}
```

서버/server_sess.c

```c
// 서버 개방
void server_sess_open(server_sess_t *sess)
{
  // server_sess_t의 주소 정보로 서버 개방 시도
  if (bind(sess->fd, (struct sockaddr *)&sess->address, sizeof(sess->address)) < 0)
  {
    PANIC("Error during binding the socket (bind)");
  }
  sess->addrlen = sizeof(sess->address);

  if (listen(sess->fd, 3) < 0)
  {
    PANIC("Error during starting listening on the socket (listen)");
  }

  printf("Simple Chat Server started on port %d\n", sess->port);
}
```

서버/server_sess.c

```c
// 서버의 메인 루프 사이클
void server_sess_update(server_sess_t *sess)
{
  // 1. 파일 디스크립터(FD) 셋 초기화 및 감시 대상 설정
  FD_ZERO(&sess->readfds);            // 읽기 감시용 FD 셋을 0으로 초기화
  FD_SET(sess->fd, &sess->readfds);   // 서버의 리스닝 소켓을 감시 대상에 추가
  sess->sd_cnt = sess->fd;            // 가장 큰 소켓 번호를 서버 소켓으로 초기 설정

  // 현재 연결된 모든 클라이언트 소켓을 감시 대상에 추가
  for (int i = 0; i < SERVER_MAX_CLIENT_CONNECTIONS; i++)
  {
    int sd = sess->clients[i].socket;
    if (sd > 0)
      FD_SET(sd, &sess->readfds);
    if (sd > sess->sd_cnt)
      sess->sd_cnt = sd;
  }
  // 활동이 있는 소켓 감시 (블로킹 상태)
  // 세 번째, 네 번째, 다섯 번째 인자가 NULL이므로 읽기 이벤트만 무한정 대기
  sess->__activity = select(sess->sd_cnt + 1, &sess->readfds, NULL, NULL, NULL);
  if (sess->__activity < 0) PANIC("Error during select operation (select)");
```

서버/server_sess.c

```
...
  // 새로운 클라이언트 연결 요청 처리
  // 서버 리스닝 소켓에 읽기 이벤트가 발생했다면 새로운 접속이 있음을 의미
  if (FD_ISSET(sess->fd, &sess->readfds))
  {
    if ((sess->__new_socket = accept(sess->fd, (struct sockaddr *)&sess->address,
(socklen_t *)&sess->addrlen)) < 0) PANIC();
    // 빈 클라이언트 슬롯을 찾아 새로운 소켓 정보 저장
    for (int i = 0; i < SERVER_MAX_CLIENT_CONNECTIONS; i++)
    { if (sess->clients[i].socket == 0) {
        sess->clients[i].socket = sess->__new_socket;
        printf("New connection: Guest%d\n", i);

        char welcome[STRING_BUFFER_SIZE];
        snprintf(welcome, sizeof(welcome), "Welcome! You are Guest%d. Use '/nick
<name>' to change nickname.\n", i);
        send_to_socket(sess->__new_socket, welcome);
        break;
  }}}
...
```

서버/server_sess.c

```
...
  // 기존 클라이언트들로부터의 메시지 수신 처리
  for (int i = 0; i < SERVER_MAX_CLIENT_CONNECTIONS; i++)
  {
    int sd = sess->clients[i].socket;
    // 해당 클라이언트 소켓에 읽기 이벤트가 발생했는지 확인
    if (FD_ISSET(sd, &sess->readfds))
    {
      memset(sess->__str_buf, 0, STRING_BUFFER_SIZE);
      // 데이터 읽기 시도
      if ((sess->__valread = read(sd, sess->__str_buf, STRING_BUFFER_SIZE)) == 0) {
        close(sd); sess->clients[i].socket = 0;
        printf("User disconnected: %s\n", sess->clients[i].nickname);
      } else {
        if (sess->__valread >= STRING_BUFFER_SIZE)
          sess->__str_buf[STRING_BUFFER_SIZE - 1] = '\0';
        else
          sess->__str_buf[sess->__valread] = '\0';
        server_sess_handle_client_message(sess, i, sess->__str_buf);
}}}}
```

서버/server_sess.c

```c
// 개방할 포트 설정
void server_sess_set_port(server_sess_t *sess, int port)
{
  sess->address.sin_port = htons(port);
  sess->port = port;
}
```

```c
// 서버 전역에 메시지 발송
void server_sess_broadcast(server_sess_t *sess, int sender_idx, const char *msg)
{
  for (int i = 0; i < SERVER_MAX_CLIENT_CONNECTIONS; i++)
    if (sess->clients[i].socket != 0 && i != sender_idx)
      send_to_socket(sess->clients[i].socket, msg);
}
```

서버/server_sess.c

```c
// 클라이언트로부터 메시지 수신 시
void server_sess_handle_client_message(server_sess_t *sess, int index, char *buffer)
{
  buffer[strcspn(buffer, "\r\n")] = 0;

  if (strlen(buffer) == 0) return;

  // 디버깅 목적으로 명령어 기능(사용자 이름 변경) 기능 추가
  if (COMMAND_NICK_IS_TRIGGERED(buffer))
  {
    command_serv_handle_nick(sess, index, buffer); return;
  }

  char msg_out[STRING_BUFFER_SIZE + 50];
  snprintf(msg_out, sizeof(msg_out), "[%s]: %s\n", sess->clients[index].nickname,
buffer);

  SERVER_SESS_LOG_AND_BRODCAST(sess, index, msg_out);
}
```

서버/빌드

- 구현체는 CMake를 이용해 빌드

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.20)
project(simple_irc_server C)
set(CMAKE_C_STANDARD 11)
set(CMAKE_C_STANDARD_REQUIRED ON)
set(HEADERS
  src/command.h     src/server_sess.h   src/socket.h
  include/client.h  include/defines.h   include/panic.h
)
set(SOURCES
  src/command.c src/server_sess.c src/socket.c src/main.c
)
add_executable(${PROJECT_NAME} ${SOURCES} ${HEADERS})
target_include_directories(${PROJECT_NAME} PRIVATE include)
```

서버-클라이언트 공통 요소

client.h

```
struct client_desc {
  int socket;
  char nickname[32];
} typedef client_desc_t;
```

서버-클라이언트 공통 요소

panic.h

```
#ifndef _SIMPLE_IRC_PANIC_H_
#define _SIMPLE_IRC_PANIC_H_

#define PANIC(msg) { \
  perror(msg); \
  exit(EXIT_FAILURE); \
}

#endif
```

클라이언트/main.c

```c
int main(int argc, char const **argv)
{
  // 클라이언트 세션 객체 생성
  client_sess_t *client_sess;
  client_sess = calloc(1, sizeof(client_sess_t));
  if (!client_sess) PANIC("Failed to allocate client session");
  init_client_sess(client_sess);

  // 서버 접속 시도
  client_sess_connect(client_sess);

  // 서버 접속 시 루프
  while (1) client_sess_update(client_sess);

  return 0;
}
```

클라이언트/client_sess.h

client_sess.h

```
struct client_sess
{
  int sock;                 // 소켓 디스크립터
  struct sockaddr_in addr; // 소켓 주소 구조체
  char nickname[32];        // 닉네임 기능에 대응
  fd_set readfds;

  int __valread;
  char __str_buf[];
} typedef client_sess_t;
```

클라이언트/client_sess.h

```c
// 생성자
void init_client_sess(client_sess_t *sess);

// 현재 사용자 이름 변경
void client_sess_set_nickname(client_sess_t *sess, const char *nickname);

// setter: 접속 대상 주소 변경
void client_sess_set_addr(client_sess_t *sess, const char *addr, int port);

// 서버 연결
void client_sess_connect(client_sess_t *sess);

// 메인 루프
void client_sess_update(client_sess_t *sess);

// 서버 연결 해제 시 행동 정의
void client_sess_on_disconnected(client_sess_t *sess);

// 서버 연결 해제
void client_sess_disconnect(client_sess_t *sess);
```

클라이언트/client_sess.c

```c
// 클라이언트 세션 생성자
void init_client_sess(client_sess_t *sess)
{
  sess->sock = -1;
  client_sess_set_nickname(sess, "Guest");

  if ((sess->sock = socket(AF_INET, SOCK_STREAM, 0)) < 0)
    PANIC("Error during socket creation (socket)");

  sess->addr.sin_family = AF_INET;
  client_sess_set_addr(sess, "127.0.0.1", SERVER_PORT);
}
```

클라이언트/client_sess.c

```c
// setter: 사용자 이름 설정
void client_sess_set_nickname(client_sess_t *sess, const char *nickname)
{
  memset(sess->nickname, 0, sizeof(sess->nickname));
  strncpy(sess->nickname, nickname, sizeof(sess->nickname) - 1);
}
```

클라이언트/client_sess.c

```c
// setter: 접속 대상 주소 설정
void client_sess_set_addr(client_sess_t *sess, const char *addr, int port)
{
  sess->addr.sin_port = htons(port);

  if (inet_pton(AF_INET, addr, &sess->addr.sin_addr) <= 0)
    PANIC("Invalid address. Address not supported (inet_pton)");
}
```

클라이언트/client_sess.c

```c
// 서버 연결
void client_sess_connect(client_sess_t *sess)
{
  if (connect(sess->sock, (struct sockaddr *)&sess->addr, sizeof(sess->addr)) < 0)
    PANIC("Connection Failed (connect)");

  printf("Connected to server!\n");
  printf("Commands: /nick <name> to change nickname.\n");
}
```

클라이언트/client_sess.c

```c
// 클라이언트 메인 루프
void client_sess_update(client_sess_t *sess)
{
  // 파일 디스크립터 셋 초기화 및 감시 대상 설정
  FD_ZERO(&sess->readfds);
  FD_SET(STDIN_FILENO, &sess->readfds);
  FD_SET(sess->sock, &sess->readfds);

  // 소켓 활동 감시 (블로킹 상태)
  if (select(sess->sock + 1, &sess->readfds, NULL, NULL, NULL) < 0)
  {
    PANIC("Error during select operation (select)");
  }
  ...
```

클라이언트/client_sess.c

```c
// 메시지 수신 시
if (FD_ISSET(sess->sock, &sess->readfds))
{
  memset(sess->__str_buf, 0, STRING_BUFFER_SIZE);
  sess->__valread = read(sess->sock, sess->__str_buf, STRING_BUFFER_SIZE);

  // 서버로부터 연결 해제 시 처리
  if (sess->__valread == 0)
  {
    client_sess_on_disconnected(sess);
    return;
  }

  // 수신된 메시지 출력
  printf("%s", sess->__str_buf);
}
...
```

클라이언트/client_sess.c

```
  ...
  // 메시지 입력 처리
  if (FD_ISSET(0, &sess->readfds))
  {
    memset(sess->__str_buf, 0, STRING_BUFFER_SIZE);
    // 표준 입력으로부터 메시지 읽기
    if (fgets(sess->__str_buf, STRING_BUFFER_SIZE, stdin) != NULL)
    {
      send(sess->sock, sess->__str_buf, strlen(sess->__str_buf), 0);
    }
  }
}
```

클라이언트/client_sess.c

```c
// 서버로부터 연결 해제 시 행동 정의
void client_sess_on_disconnected(client_sess_t *sess)
{
  printf("Disconnected from server.\n");
  exit(EXIT_SUCCESS);
}
```

```c
// 서버 연결 해제
void client_sess_disconnect(client_sess_t *sess)
{
  // 파일 디스크립터 닫기
  close(sess->sock);
}
```

실행 결과

```
./simple_irc_client
❯ ./simple_irc_client
Connected to server!
Commands: /nick <name> to change nickname.
Welcome! You are Guest1. Use '/nick <name>' to change nickname.
ぷにぷに
[Guest0]: ぷにぷにって言うな！
[Guest0]: おい！赤スーパーだったからって、ぷにぷにって言えば許されるとでも思ってるのか！他に言うこ
とがあるだろ！
[Guest0]: おいなんかいうことがあったろ
…でもかわいい
[Guest0]: あ、いいねいいね
[Guest0]: 気分がよくなってきた
かわいいでもぷにぷに…
[Guest0]: …何か意味違うそう
```

실행 결과

./simple_irc_server

```
❯ ./simple_irc_server
Server session initialized.
Starting server on port 6667...
Simple Chat Server started on port 6667
New connection: Guest0
New connection: Guest1
[Server] Guest0 changed name to Duda
[Server] Guest1 changed name to John Kirby
[Duda]: Korea is helpful in Ukraine's war against Russia bec
ause of its ammunition deliveries, as its huge supply of art
illery shells.
[Duda]: I didn't think this was "going to be possible withou
t the intervention of the United States," as Korea is "fearf
ul" of Russian and Chinese reactions.
[Duda]: US have to intervene with Korea on Ukrainian arms is
sue
[John Kirby]: We have a treaty alliance with South Korea.
[John Kirby]: We take that commitment very, very seriously.
The U.S. is grateful for the support South Korea has already
 provided to Ukraine.
```

```
❯ ./simple_irc_client
Connected to server!
Commands: /nick <name> to change nickname.
Welcome! You are Guest0. Use '/nick <name>' to change nickna
me.
/nick Duda
[Server] Nickname changed to Duda
[Server] Guest1 changed name to John Kirby
Korea is helpful in Ukraine's war against Russia because of
its ammunition deliveries, as its huge supply of artillery s
hells.
I didn't think this was "going to be possible without the in
tervention of the United States," as Korea is "fearful" of R
ussian and Chinese reactions.
US have to intervene with Korea on Ukrainian arms issue
[John Kirby]: We have a treaty alliance with South Korea.
[John Kirby]: We take that commitment very, very seriously.
The U.S. is grateful for the support South Korea has already
 provided to Ukraine.
```

```
❯ ./simple_irc_client
Connected to server!
Commands: /nick <name> to change nickname.
Welcome! You are Guest1. Use '/nick <name>' to change nickna
me.
[Server] Guest0 changed name to Duda
/nick John Kirby
[Server] Nickname changed to John Kirby
[Duda]: Korea is helpful in Ukraine's war against Russia bec
ause of its ammunition deliveries, as its huge supply of art
illery shells.
[Duda]: I didn't think this was "going to be possible withou
t the intervention of the United States," as Korea is "fearf
ul" of Russian and Chinese reactions.
[Duda]: US have to intervene with Korea on Ukrainian arms is
sue
We have a treaty alliance with South Korea.
We take that commitment very, very seriously. The U.S. is gr
ateful for the support South Korea has already provided to U
kraine.
^[
```

실행 결과

./simple_irc_server

```
❯ ./simple_irc_client
zsh: no such file or directory: ./simple_irc_client
❯ ./simple_irc_server
Server session initialized.
Starting server on port 6667...
Simple Chat Server started on port 6667
New connection: Guest0
New connection: Guest1
[Guest0]: 저 뭐 하나만 여쭤봐도 되겠습니까?
[Guest1]: 네네네
[Guest0]: 저희가 이걸 좀 촉박하게 제안을 드렸잖아요?
[Guest1]: 네네
[Guest0]: 그런데 오늘 보니까 PPT가 70장이 넘었는데
[Guest0]: 자..혹시.. 평소에도 이런 생각하고 계신가요?
[Guest0]: 급하게 만들었다기에는 너무나도 일관성있고 해서
[Guest0]: 혹시 이게 압둘씨께서 평상시에 이런걸 막 미리 준비
하신게 아닐까
[Guest1]: 일관성이 있다라는 것은 무엇인가?
[Guest1]: 이 모든 내용이 참이라는 것을 증명하는 것 아니겠습
니까?
[Guest0]: 아..;;
[Guest0]: 방송인이 되어주셔서 정말 감사합니다..;;
[Guest0]: 사이비 종교같은거 안 만들어주셔서 정말 감사합니다;
;
[Guest1]: 예예 항상 감사하십시오
```

```
❯ ./simple_irc_client
Connected to server!
Commands: /nick <name> to change nickname.
Welcome! You are Guest0. Use '/nick <name>' to change nickna
me.
저 뭐 하나만 여쭤봐도 되겠습니까?
[Guest1]: 네네네
저희가 이걸 좀 촉박하게 제안을 드렸잖아요?
[Guest1]: 네네
그런데 오늘 보니까 PPT가 70장이 넘었는데
자..혹시.. 평소에도 이런 생각하고 계신가요?
급하게 만들었다기에는 너무나도 일관성있고 해서
혹시 이게 압둘씨께서 평상시에 이런걸 막 미리 준비하신게 아닐
까
[Guest1]: 일관성이 있다라는 것은 무엇인가?
[Guest1]: 이 모든 내용이 참이라는 것을 증명하는 것 아니겠습
니까?
아..;;
방송인이 되어주셔서 정말 감사합니다..;;
사이비 종교같은거 안 만들어주셔서 정말 감사합니다;;
[Guest1]: 예예 항상 감사하십시오
```

```
❯ ./simple_irc_client
Connected to server!
Commands: /nick <name> to change nickname.
Welcome! You are Guest1. Use '/nick <name>' to change nickna
me.
[Guest0]: 저 뭐 하나만 여쭤봐도 되겠습니까?
네네네
[Guest0]: 저희가 이걸 좀 촉박하게 제안을 드렸잖아요?
네네
[Guest0]: 그런데 오늘 보니까 PPT가 70장이 넘었는데
[Guest0]: 자..혹시.. 평소에도 이런 생각하고 계신가요?
[Guest0]: 급하게 만들었다기에는 너무나도 일관성있고 해서
[Guest0]: 혹시 이게 압둘씨께서 평상시에 이런걸 막 미리 준비
하신게 아닐까
일관성이 있다라는 것은 무엇인가?
이 모든 내용이 참이라는 것을 증명하는 것 아니겠습니까?
[Guest0]: 아..;;
[Guest0]: 방송인이 되어주셔서 정말 감사합니다..;;
[Guest0]: 사이비 종교같은거 안 만들어주셔서 정말 감사합니다;
;
예예 항상 감사하십시오
```